CampusEvents

Campus Event & Ticket Booking System

COP 4710 — Database Systems University of South Florida Spring 2026

Intermediate Report 2 — Due April 10, 2026

Contents

1. Team Roster

This report is submitted by the two-person team listed in the mini report. Per Dr. Tu's course guidelines, two-person groups carry the same deliverable expectations as a three-person group, and the scope demonstrated in this progress report reflects that.

Name	Email	Role
Trevor Flahardy	trevorflahardy@usf.edu	Developer
Sofia Cobo Navas	scobonavas@usf.edu	Developer

2. Purpose of This Report

The April 10 milestone asks us to show that an **initial version of all three tiers** of the proposed system — interface, application logic, and database — is in a working state. Functionality can be of the simplest form; the grading criterion is whether the system is demonstrably alive end-to-end.

We are pleased to report that CampusEvents is already well past the "simplest form" bar. Every tier is running, the three tiers communicate end-to-end over HTTP and SQL, all ten planned SQL query types from the mini report are implemented against real endpoints, and the user interface exercises those endpoints through a React single-page application. This document walks through each tier, cites the files where the code lives, shows representative code excerpts, and includes screenshots of the running system.

> *All code references in this report point to files inside the project repository. The repo lives at the path printed in the README and is structured as* `backend/` *(application + data access),* `frontend/` *(interface),* `schema.sql` *(reference DDL), and* `docker-compose.yml` *(orchestration). The system is launched with a single command:* `docker compose up`*.*

3. Three-Tier Architecture Overview

The system follows a textbook three-tier architecture. The **interface tier** is a React 19 single-page application served by Vite; it renders the UI, manages local state, and talks to the backend over `fetch`. The **application tier** is a TypeScript REST API running on the Bun runtime, built with the Hono framework; it validates input, enforces authorization, and issues parameterized SQL through the `postgres.js` driver. The **data tier** is a PostgreSQL 16 database running in Docker, owning all persistent state and enforcing integrity constraints at the schema level.

Tier	Technology	Port	Responsibility
Interface	React 19 + Vite + Tailwind v4	:5173	Rendering, routing, client-side state
Application	Bun + Hono (REST, TypeScript)	:3000	Validation, auth, SQL queries via postgres.js
Data	PostgreSQL 16 (Docker)	:5432	Storage, constraints, referential integrity

Figure 1: Three-tier architecture — browser calls the Hono API, which issues parameterized SQL to PostgreSQL via the postgres.js driver.

> *postgres.js plays the role of JDBC/ODBC in this stack: a lightweight PostgreSQL client for JavaScript that sends parameterized SQL directly to the database. All queries are written as tagged template literals (e.g. sql`SELECT * FROM users WHERE id = ${id}`), which automatically escape parameters to prevent SQL injection. The course requirement to "utilize JDBC/ODBC or other communication protocols to connect to the database and send in queries" is satisfied through this driver.*

4. Data Tier — PostgreSQL + Raw SQL

4.1. Running Database

A PostgreSQL 16 instance runs inside Docker, provisioned by docker-compose.yml at the project root. On first boot, the container executes schema.sql as part of its init script, creating the enums and tables laid out in the mini report. The service listens on localhost:5432 with database campusevents, user campus, password campus123.

```
// docker-compose.yml (excerpt)
postgres:
  image: postgres:16-alpine
  container_name: campusevents_db
  environment:
    POSTGRES_USER: campus
    POSTGRES_PASSWORD: campus123
    POSTGRES_DB: campusevents
  ports: ["5432:5432"]
  volumes:
```

```
    - postgres_data:/var/lib/postgresql/data
    - ./schema.sql:/docker-entrypoint-initdb.d/01-schema.sql:ro
```

4.2. Schema (Raw SQL)

The authoritative schema definition lives in `schema.sql` at the project root as standard PostgreSQL DDL. This file defines all enums, tables, constraints, indexes, and sample data. At backend startup, the server applies `schema.sql` directly using `postgres.js`'s `unsafe()` method. All statements use `IF NOT EXISTS` / `ON CONFLICT DO NOTHING` guards, making the file idempotent and safe to run on every boot.

```
-- schema.sql (excerpt)
CREATE TABLE IF NOT EXISTS events (
  id          SERIAL        PRIMARY KEY,
  title       VARCHAR(200)  NOT NULL,
  description TEXT,
  location    VARCHAR(200)  NOT NULL,
  start_time  TIMESTAMP     NOT NULL,
  end_time    TIMESTAMP     NOT NULL,
  capacity    INTEGER       NOT NULL CHECK (capacity > 0),
  ticket_price NUMERIC(10, 2) NOT NULL DEFAULT 0.00
                            CHECK (ticket_price >= 0),
  status      event_status  NOT NULL DEFAULT 'upcoming',
  organizer_id INTEGER      NOT NULL REFERENCES users(id)
                            ON DELETE CASCADE,
  created_at  TIMESTAMP     NOT NULL DEFAULT NOW(),
  CONSTRAINT valid_time_range CHECK (end_time > start_time)
);
```

4.3. Tables & Seed Data

All five tables from the mini report are live: `users`, `events`, `tickets`, `categories`, and `event_categories`. A sixth table, `images`, stores uploaded event banners as base-64 blobs (a small addition beyond the mini report so organizers can attach artwork to events). A seed script at `backend/src/db/seed.ts` populates the database with representative users (a student, an organizer, an admin), a handful of events across categories, and a sample ticket — enough content for the interface tier to render meaningful data on first load.

```
futur@TrevMacbook project_4 % docker exec -it campusevents_db psql -U campus -d campusevents
psql (16.13)
Type "help" for help.

campusevents=# \dt
             List of relations
 Schema |       Name        | Type  | Owner
--------+-------------------+-------+--------
 public | categories        | table | campus
 public | event_categories  | table | campus
 public | events            | table | campus
 public | images            | table | campus
 public | tickets           | table | campus
 public | users             | table | campus
(6 rows)

campusevents=#
```

Figure 2: PostgreSQL 16 running inside Docker after `docker compose up`. Output of `\dt` in `psql` shows the six relations created by `schema.sql` — the five from the ER diagram, plus `images` for uploaded event banners and profile photos.

4.4. Integrity Constraints

The constraints promised in the mini report are enforced at the schema level. Foreign keys link events.organizer_id → users.id, tickets.user_id → users.id, tickets.event_id → events.id, and the join table's composite primary key on `(event_id, category_id)`. The `UNIQUE(user_id, event_id)` constraint on `tickets` prevents double bookings — if a user tries to purchase a second ticket for the same event, PostgreSQL raises SQL state `23505` and the API translates that into a `409 Conflict` response to the client.

```ts
// backend/src/routes/tickets.ts --- catching unique-constraint violation
try {
  const [inserted] = await sql`
    INSERT INTO tickets (user_id, event_id, confirmation_code)
    VALUES (${userId}, ${eventId}, ${confirmationCode})
    RETURNING *
  `;
  return c.json(inserted, 201);
} catch (err) {
  const pgErr = err as { code?: string };
  if (pgErr.code === "23505") {
    return c.json({ error: "You already have a ticket for this event" }, 409);
  }
  throw err;
}
```

5. Application Tier — Bun + Hono REST API

5.1. Server Entry Point

The backend is a TypeScript application that runs on the Bun runtime. Its entry point is `backend/src/index.ts`, which applies `schema.sql` on startup, then creates a Hono `App` with logger and CORS middleware, mounts five sub-routers under `/api/*`, and exposes a `/health` probe. All database queries use `postgres.js` tagged template literals. Bun's `--hot` flag gives us hot reload during development; the Docker service reuses the same `bun run src/index.ts` command in production.

```
// backend/src/index.ts (excerpt)
const app = new Hono();
app.use("*", logger());
app.use("/api/*", cors({ origin: ["http://localhost:5173"] }));

app.route("/api/auth",       authRouter);
app.route("/api/categories", categoriesRouter);
app.route("/api/events",     eventsRouter);
app.route("/api/tickets",    ticketsRouter);
app.route("/api/users",      usersRouter);

console.log(`🚀 Server running on http://localhost:3000`);
export default { port: 3000, fetch: app.fetch };
```

5.2. Route Modules

The API is split into six route modules under `backend/src/routes/`, each owning a resource:

Module	Mounted at	Purpose
`auth.ts`	`/api/auth`	Register, login (bcrypt + JWT), `GET /me`
`users.ts`	`/api/users`	Profile CRUD, role management
`events.ts`	`/api/events`	Event CRUD, listing, stats, attendees
`tickets.ts`	`/api/tickets`	Purchase, list, cancel, check-in
`categories.ts`	`/api/categories`	Category lookup, popular categories
`images.ts`	`/uploads`	Event banner upload & retrieval

5.3. Authentication & Authorization

We implemented the first two bonus features from the assignment handout — **user accounts with userID/password** and **different privileges for different users** — directly in the application tier. Passwords are hashed with bcrypt (`@node-rs/bcrypt`) on registration and verified on login. A successful login returns a JWT signed with `HS256` containing the user's `id` and `role`. A Hono middleware at `backend/src/middleware/auth.ts` verifies the token on protected routes and injects `userId` and `userRole`

into the request context; a `requireRole(...)` factory enforces role gating on organizer- and admin-only endpoints.

```
// backend/src/middleware/auth.ts (excerpt)
export async function authMiddleware(c: Context<AuthEnv>, next: Next) {
  const header = c.req.header("Authorization");
  if (!header?.startsWith("Bearer ")) {
    return c.json({ error: "Missing or invalid Authorization header" }, 401);
  }
  const payload = await verify(header.slice(7), JWT_SECRET, "HS256");
  c.set("userId",   payload.sub  as number);
  c.set("userRole", payload.role as string);
  await next();
}

export function requireRole(...roles: string[]) {
  return async (c: Context<AuthEnv>, next: Next) => {
    if (!roles.includes(c.get("userRole"))) {
      return c.json({ error: "Forbidden" }, 403);
    }
    await next();
  };
}
```

5.4. Input Validation

Every write endpoint validates its JSON body with a Zod schema before touching the database. This catches malformed input at the edge of the application tier — the database never sees an event with a missing title or a ticket with a negative `event_id`.

```
// backend/src/routes/tickets.ts
const purchaseSchema = z.object({
  userId:  z.number().int().positive(),
  eventId: z.number().int().positive(),
});
const parsed = purchaseSchema.safeParse(await c.req.json());
if (!parsed.success) return c.json({ error: parsed.error.flatten() }, 400);
```

6. Interface Tier — React + Vite + Tailwind

6.1. Page Inventory

The frontend is a React 19 single-page application bootstrapped with Vite and styled with Tailwind CSS v4. It ships eight routed pages — three more than the mini report promised — giving us comfortable

margin above the "at least three pages" requirement. Routing is handled by `react-router-dom` v7; protected routes redirect unauthenticated users to `/login`.

Page (`frontend/src/pages/`)	Route	Purpose
`Landing.tsx`	`/`	Marketing-style landing page for logged-out visitors
`BrowseEvents.tsx`	`/events`	Event discovery — search, category filter, date range
`EventDetail.tsx`	`/events/:id`	Full event info, remaining capacity, Book Ticket button
`Login.tsx`	`/login`	JWT-based sign-in form
`Register.tsx`	`/register`	Account creation with role selection
`MyTickets.tsx`	`/my-tickets`	Authenticated student's booking history & check-in
`OrganizerDashboard.tsx`	`/dashboard`	Create/edit/cancel events, view attendees
`AdminPanel.tsx`	`/admin`	Full system view — manage users, events, categories

6.2. Client ↔ API Contract

All network calls are funneled through a small typed client at `frontend/src/lib/api.ts`. Each method returns a strongly-typed result that mirrors a row shape from the backend, so the interface tier never hand-crafts a URL or parses a raw response. Authentication state lives in `frontend/src/context/AuthContext.tsx`: on login the JWT is persisted to `localStorage`, restored on page reload, and attached to subsequent requests via the `Authorization: Bearer` header.

```
// frontend/src/pages/BrowseEvents.tsx --- hitting GET /api/events
useEffect(() => {
  const timer = setTimeout(async () => {
    setLoading(true);
    const params: Record<string, string> = {};
    if (search)     params.search     = search;
    if (categoryId) params.categoryId = categoryId;
    if (from)       params.from       = from;
    if (to)         params.to         = to;
    if (status)     params.status     = status;
    const data = await api.getEvents(params);
    setEvents(data);
    setLoading(false);
  }, 300);            // debounce
  return () => clearTimeout(timer);
}, [search, categoryId, from, to, status]);
```

6.3. Screenshots of the Running Interface

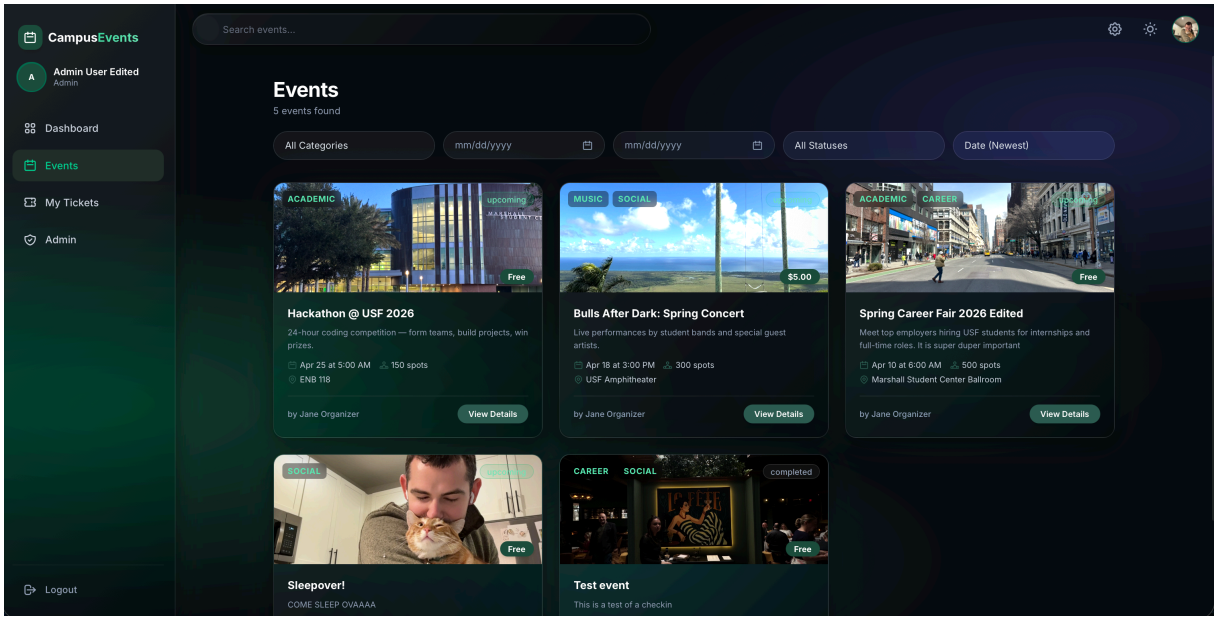

Figure 3: **Browse Events** (`/events`) — grid of event cards fetched from `GET /api/events`, with live search, category filter, and date-range picker. This view exercises queries Q1 (`SELECT + JOIN` on events + users) and Q10 (`BETWEEN` date filtering) on every keystroke.

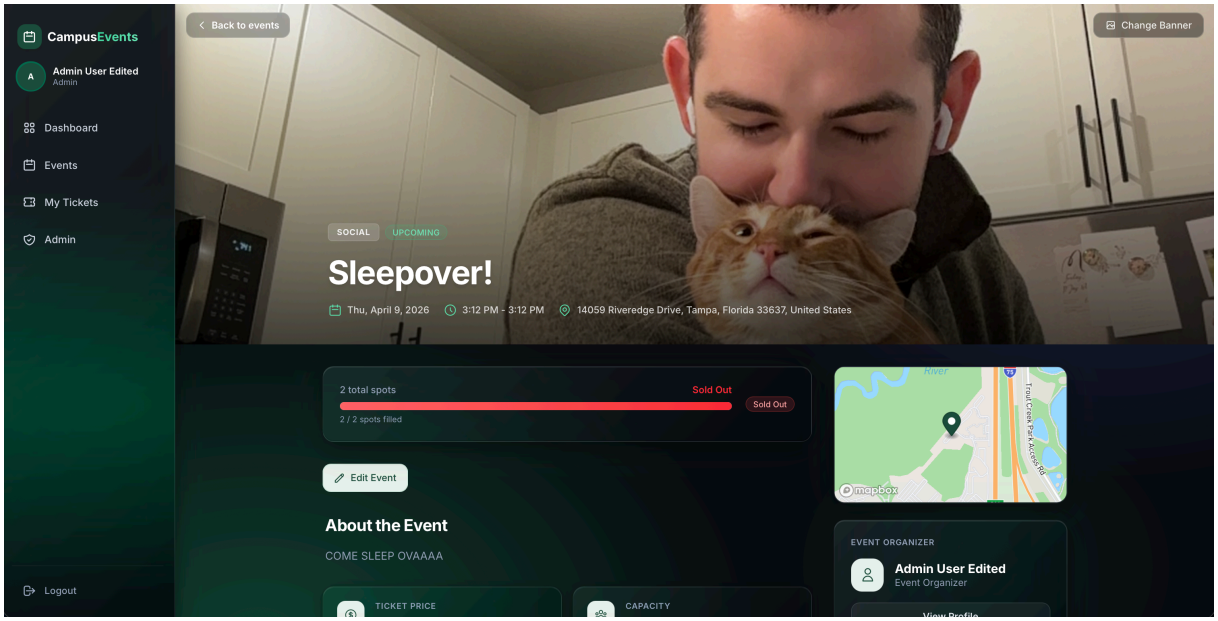

Figure 4: **Event Detail** (`/events/:id`) — hero banner, description, remaining-capacity counter, and Book Ticket action. The remaining-capacity number comes from query Q3 (`SELECT` with subquery) computed on the backend.

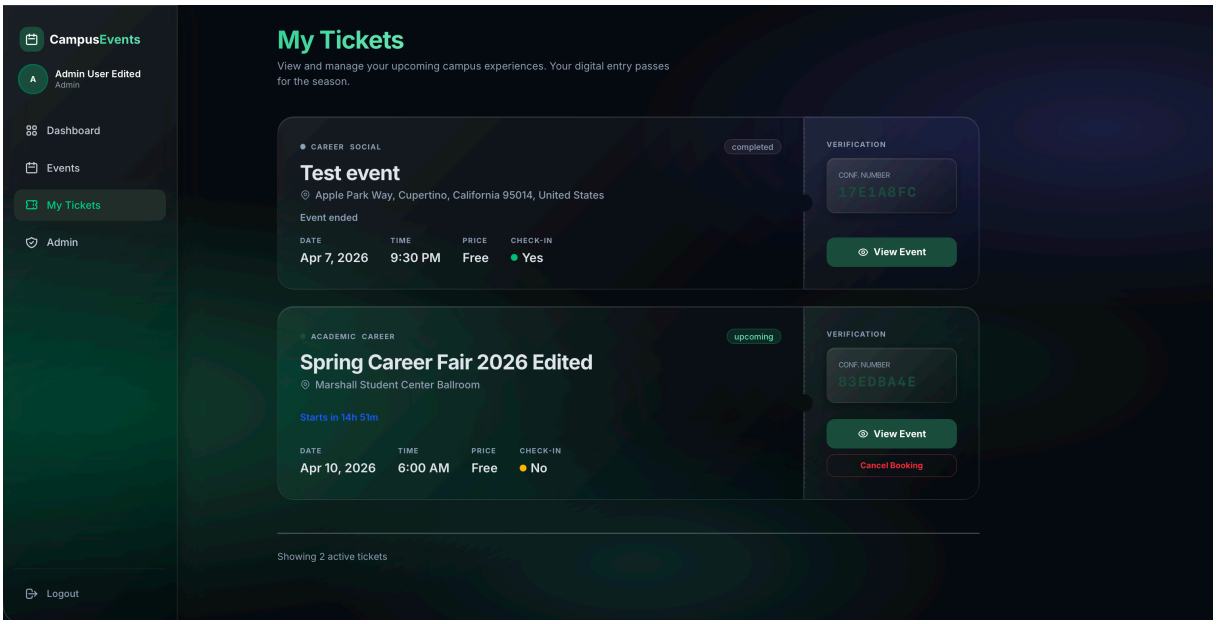

Figure 5: **My Tickets** (`/my-tickets`) — authenticated student's bookings with confirmation codes and check-in controls, backed by query Q4 (multiple `JOIN`s across tickets, events, and categories).

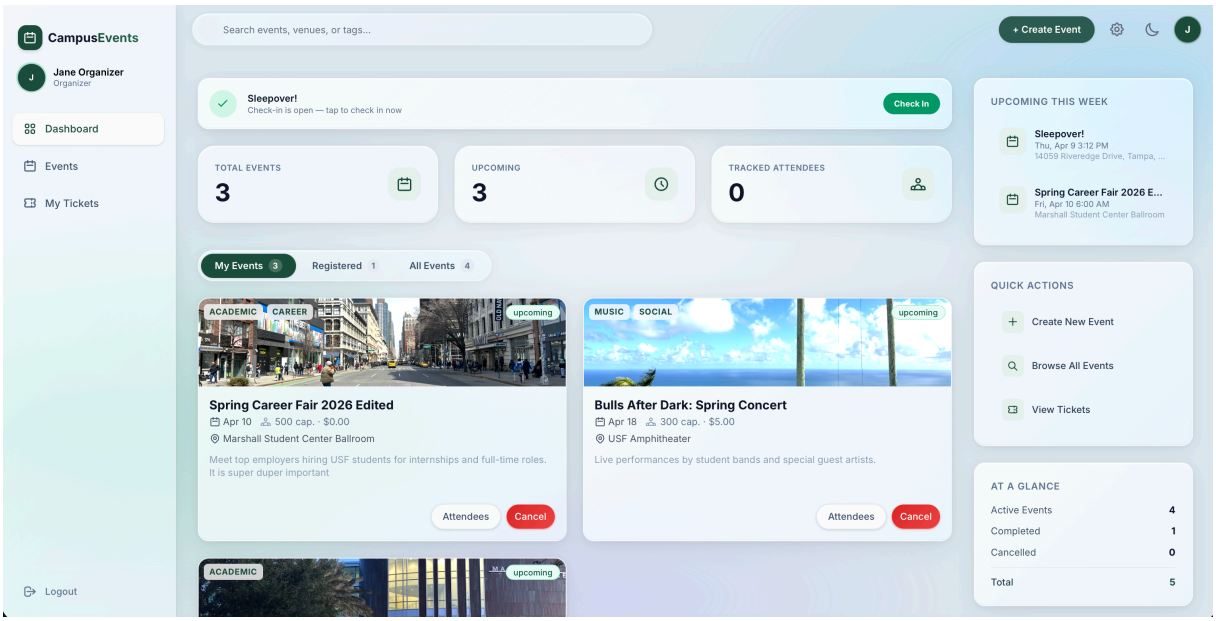

Figure 6: **Organizer Dashboard** (`/dashboard`) — event creation, edits, attendee list, and sales counts from query Q2 (`GROUP BY` + `COUNT`). The Cancel action issues the `UPDATE` from Q7.

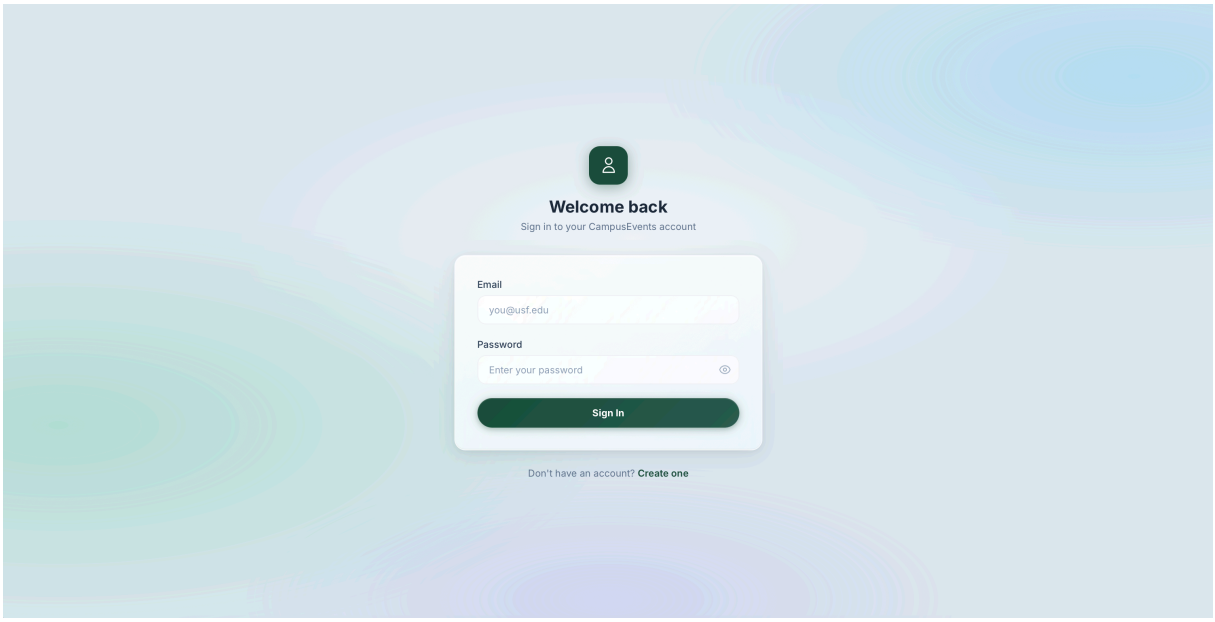

Figure 7: **Login** (`/login`) — submits credentials to `POST /api/auth/login`, receives a JWT, and stores it in `localStorage` via `AuthContext`.

7. End-to-End Walkthrough — Ticket Purchase

To make it concrete that the three tiers actually talk to each other, this section traces a single user action — **a student clicking "Book Ticket" on an event** — through every layer. This is the canonical Q5 path (`INSERT` into `tickets`).

1. **Interface tier.** The user is on `EventDetail.tsx`. Clicking the button calls `api.purchaseTicket(user.id, event.id)`, which issues `POST /api/tickets` with the JWT attached.
2. **Application tier — validation.** Hono routes the request into `tickets.ts`. The `authMiddleware` verifies the JWT and sets `userId` / `userRole`. The body is parsed with `purchaseSchema` (Zod) to guarantee both IDs are positive integers.
3. **Application tier — business rules.** The handler issues a pre-flight `SELECT` against `events` with a correlated sub-select to compute `ticketsSold`. If the event is cancelled or sold out, the API short-circuits with a `400` response; the interface tier shows a toast.
4. **Data tier — INSERT.** If the pre-flight passes, the handler generates an 8-character confirmation code and issues `INSERT INTO tickets`. The `UNIQUE(user_id, event_id)` constraint is the final guard: if the student already holds a ticket, PostgreSQL raises `23505` and the API returns `409 Conflict`.
5. **Response path.** On success, the new ticket row is returned as JSON. The frontend shows a success toast via `sonner` and navigates to `/my-tickets`, where a fresh call to `GET /api/tickets/user/:userId` re-renders the list including the new booking.

> *This walkthrough touches queries Q3 (capacity sub-select), Q5 (INSERT), and Q4 (multi-JOIN re-fetch), and involves the JWT middleware on the application tier and two foreign-key plus one unique constraint on the data tier. A single button click exercises every layer of the stack.*

8. SQL Query Implementation Status

All ten SQL query types planned in the mini report are implemented as real REST endpoints driving real UI features. The table below maps each query to its application-tier handler and its interface-tier caller.

#	Type	Endpoint (handler file)	UI caller
Q1	`SELECT + JOIN`	`GET /api/events` `routes/events.ts`	`BrowseEvents.tsx`
Q2	`GROUP BY + COUNT`	`GET /api/events/stats` `routes/events.ts`	`OrganizerDashboard.tsx`
Q3	`SELECT + subquery`	`GET /api/events/:id` `routes/events.ts`	`EventDetail.tsx`
Q4	`SELECT + multi-JOIN`	`GET /api/tickets/user/:userId` `routes/tickets.ts`	`MyTickets.tsx`
Q5	`INSERT`	`POST /api/tickets` `routes/tickets.ts`	`EventDetail.tsx`
Q6	`UPDATE`	`PATCH /api/tickets/:id/checkin` `routes/tickets.ts`	`MyTickets.tsx`
Q7	`UPDATE`	`PATCH /api/events/:id` `routes/events.ts`	`OrganizerDashboard.tsx`
Q8	`DELETE`	`DELETE /api/tickets/:id` `routes/tickets.ts`	`MyTickets.tsx`
Q9	`SELECT + HAVING`	`GET /api/categories/popular` `routes/categories.ts`	`BrowseEvents.tsx`
Q10	`SELECT + BETWEEN`	`GET /api/events?from=&to=` `routes/events.ts`	`BrowseEvents.tsx`

Some representative SQL is shown below. Each snippet is the actual query the backend issues via `postgres.js` tagged templates; the comments indicate which query number and endpoint it backs.

```
// Q1 + Q10: events list with optional BETWEEN date filtering
// postgres.js fragment composition builds a safe dynamic WHERE clause.
```

```
const conditions = [sql`TRUE`];
if (from) conditions.push(sql`e.start_time >= ${new Date(from)}`);
if (to)   conditions.push(sql`e.start_time <= ${new Date(to)}`);
const where = conditions.reduce((a, c) => sql`${a} AND ${c}`);

const rows = await sql`
  SELECT e.*, u.name AS organizer_name
  FROM events e
  INNER JOIN users u ON e.organizer_id = u.id
  WHERE ${where}
  ORDER BY e.start_time
`;

// Q2: tickets sold per event (GROUP BY + COUNT)
const stats = await sql`
  SELECT e.id AS event_id, e.title,
         COUNT(t.id)::int AS tickets_sold, e.capacity
  FROM events e
  LEFT JOIN tickets t ON e.id = t.event_id
  WHERE e.status != 'cancelled'
  GROUP BY e.id, e.title, e.capacity
`;

// Q9: categories with more than one event (SELECT + HAVING)
const popular = await sql`
  SELECT c.id AS category_id, c.name,
         COUNT(ec.event_id)::int AS event_count
  FROM categories c
  INNER JOIN event_categories ec ON c.id = ec.category_id
  GROUP BY c.id, c.name
  HAVING COUNT(ec.event_id) > 1
`;
```

9. Running the System

The entire stack boots with one command. The following transcript is what a grader would see on a fresh clone:

```
$ docker compose up
[+] Running 3/3
 ✓ Container campusevents_db      Started
 ✓ Container campusevents_api     Started
 ✓ Container campusevents_web     Started
campusevents_db  | PostgreSQL init process complete; ready for connections.
campusevents_api | ✅ Migrations applied
campusevents_api | 🚀 Server running on http://localhost:3000
```

```
campusevents_web | VITE v8.0.1  ready in 412 ms
campusevents_web |  ?  Local:   http://localhost:5173/
```

Navigating to `http://localhost:5173` loads the Landing page; `http://localhost:3000/health` returns `{"status":"ok"}`; `psql -h localhost -U campus -d campusevents` opens a shell onto the live database. A seed script (`bun run src/db/seed.ts`) populates representative users, events, and tickets for demo purposes.

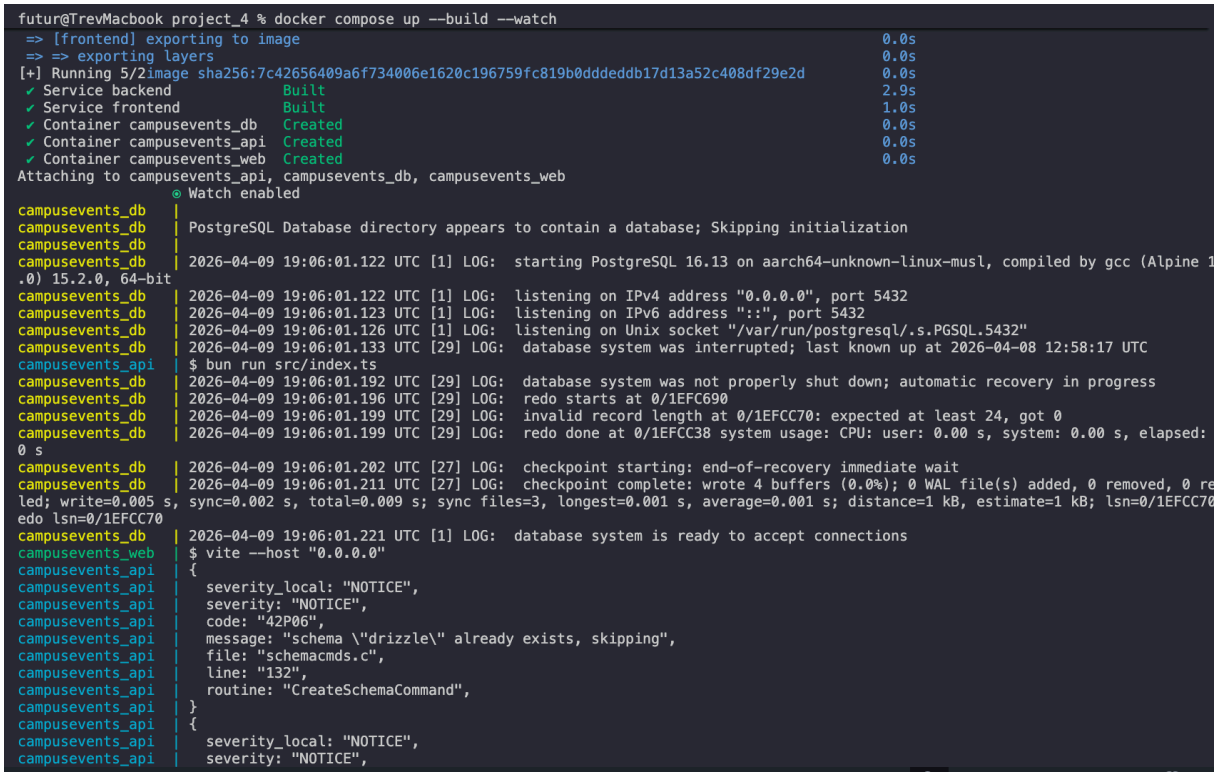

Figure 8: `docker compose up` — all three tiers booting together. The Hono API reports "Migrations applied" before accepting traffic, and Vite serves the React bundle on port 5173.

10. Status Against Course Requirements

The assignment handout lists four graded features and a set of optional bonuses. Our status on each, as of this report, is below.

#	Requirement	Status
1	DBMS-backed database with ≥ 3 relations loaded with data	✓ Done (5 tables + seed)
2	≥ 8 distinct SQL query types, including state-modifying	✓ Done (10 implemented)
3	Web-based interface with ≥ 3 distinct pages	✓ Done (8 pages)

#	Requirement	Status
4	JDBC/ODBC-style connection to send queries to the DB	✓ Done (`postgres.js` raw SQL)
—	**Bonus** — user accounts with ID/password	✓ Done (bcrypt + JWT)
—	**Bonus** — different privileges per user	✓ Done (role middleware)
—	**Bonus** — client-side scripts for application logic	✓ Done (React SPA)
—	**Bonus** — stored procedures / functions on the DB side	Planned for final
—	**Bonus** — views with per-role privileges	Planned for final

11. Remaining Work Toward May 1

The progress reported here puts us comfortably past the "simplest form" threshold for this milestone. The remaining work for the final submission and live demo is focused on depth rather than breadth:

- **Database-side logic.** Add a small set of PL/pgSQL stored procedures (e.g., atomic ticket purchase with capacity check) and a handful of `CREATE VIEW` definitions exposing role-specific projections. These pick up two of the remaining bonus points.
- **Admin panel polish.** The `AdminPanel.tsx` page currently lists users and events; we will flesh out role assignment, bulk category editing, and a simple analytics dashboard on top of Q2 and Q9.
- **Demo script.** Prepare a 10-minute guided walkthrough that hits every query type, including the state-modifying ones, and shows a deliberate failure (e.g., double-booking) to demonstrate database constraints firing.
- **Test coverage.** An integration test suite against the running API has been added using Bun's built-in test runner, covering all ten query types and key error paths. The demo environment can be verified with `bun test` before the live session.
- **Report polish.** Expand this document into the final report with a revised ER diagram, schema diff from the mini report, and a complete user manual.

> ***TL;DR.*** *All three tiers are running, communicate end-to-end, and cover every requirement on the rubric plus three of the five bonus items. All SQL queries are written directly using `postgres.js` tagged templates. The remaining work for May 1 is additive: database-side procedures, an admin panel pass, and a demo script.*